BRAC UNIVERSITY
Department of Computer Science and Engineering

CSE 321

Operating Systems

Lecture Notes

MUSFIKUR RAHMAN MAHIN
ID 23201432
Summer 2026

RESOURCES

Operating System Concepts, 10th Edition
Abraham Silberschatz, Peter B. Galvin, Greg Gagne

NOTE

These notes cover the topics included in the Summer 2026 syllabus of CSE 321. Some simulations and security topics are not included.

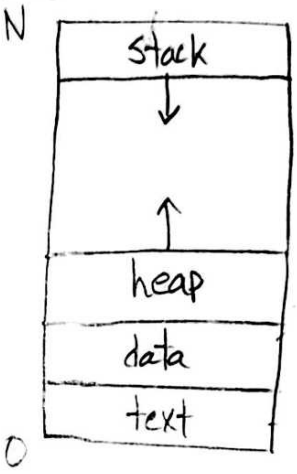
TOPIC NAME: Process DAY: _____

TIME: DATE: / /

Process

Process is a program in execution.

Memory layout of a process:



text: executable code

data: Global variable

heap: dynamically allocated memory during program run

stack: temporary data storage when invoking function such as parameters, return address, local variable.

The status of the current activity of a process is represented by the value of the "program Counter."

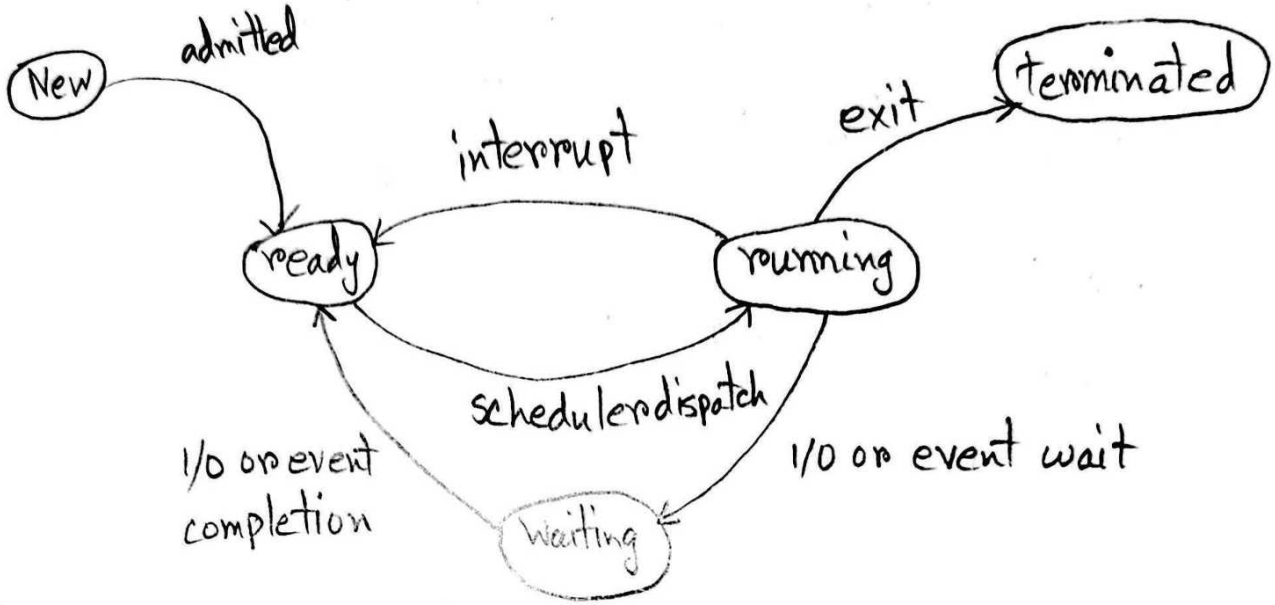
When a function is called, an "activation record" containing function parameters, local variables and returned address is pushed into the stack.

Similarly, the Heap will grow as memory.

TOPIC NAME : _____ DAY : _____

TIME : DATE : / /

state



*) Only one process can be running on any processor core at any instant.

PCB

Each process is represented in the OS by a process control block/ task control block.

P. state
P. number
P. C.
registers
memory limits
list of open files

Threads

The most basic process is a program that performs a single thread of execution.

Most modern OS allows a process to have multiple threads of execution thus can perform more than one task at a time.

Process Scheduling

The process scheduler selects an available process for program execution on a core.

The number of processes currently in memory is known as the degree of multiprogramming.

process - [I/O bound
          [cpu bound

As a process enter the system, it is put into a "Ready Queue", this queue is generally stored as a linked list.

"Wait Queue"

TOPIC NAME : _____ DAY : _____

TIME : DATE : / /

Context Switch

When an interrupt occurs, the system needs to save the current context of the process running on the CPU. The context is represented in the PCB.

Switching the cpu core requires performing a "state save" of the current process and a "state restore" of a different process. This is known as context switch.

The more complex the OS, the greater the work must be done during context switch.

Dual Mode

Two mode separate OS code and user defined code.

Mode bit determine the current mode. kernel (0), User (1)

It provides protection to the system.

Operation on Process

Processes are identified using an unique process identifier (pid) → int

"systemd" is the first user process created when the system boots with pid of 1. systemd process creates processes which provide additional services.

when fork() happens the return for that call is 0 for the new(child) process and the process identifier (non zero) of the child is returned for the parent.

```
val= fork();
```
→ inherits the address space of the parent.

Val = 0 for child
Val = n for parent ; n > 0

when a child creates another process its identity is passed to the parent.

TOPIC NAME : _____ DAY : _____

TIME : DATE : / /

Termination

A parent may terminate an execution of a child,

i. Child has exceeded usage of resources.

ii. Assigned task to the child is no longer required.

iii. The parent is exiting and the OS doesn't allow the child to continue. [cascading termination]

wait() returns the pid of child + status info & [?]

Zombie Process: When a child is finished running but the parent hasn't yet collected its exit status. Entry in process table remains. Parent did not invoke wait().

Orphan Process: A process whose parent terminated before it. The OS assign it a new parent (init/systemd) Parent terminated without wait().

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

CPU Scheduling

Os switch the CPU among processes to make the computer more productive.

Process execution consists of a "Cycle" of CPU execution and I/O wait.

The "CPU scheduler" selects a process from the memory that are ready to execute. The records in the ready queues are generally PCBs of the processes.

CPU scheduling decisions may take place under the following circumstances:

i. When a process switches from the running state to the waiting state.
ii. running state to ready state. [Ex: Interrupt]
iii. Waiting state to ready state. [Ex: I/O complete]
iv. When a process terminates.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Nonpreemptive → Process keeps the cpu until it releases it [1,4]

Preemptive → Processes share the cpu [2,3]
Most modern OS.

Preemptive scheduling can result in race conditions when data are shared among several processes.
In a case of two processes that share data. While one is updating data, it is preempted so that the second process can run. The other process then tries to read the data and finds data in an inconsistent state.
Or maybe when kernel is performing a system call, the process is preempted and kernal need to read/modify the same structure (process).

*) Before entering a critical section the OS temporarily disables interrupts.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Dispatcher → is the module that gives control of the core to the process seleted by the CPU scheduler. [invoked during every context switch]

This function,

i. Switches context from one process to another

ii. Switches to user mode.

iii. Jumps to the propere location in the user program to resume that program.

Dispatch latency → time taken to stop one process and start another.

A voluntary con.switch occurs when a p. gives up the control of cpu because it requires a nesource that is currently unavailable.

A nonvoluntary con.switch occurs when the cpu has been taken away. Ex: Time slice expired on preempted by a higher-priority process.

↑ Cpu Utilization: Percentage of time the cpu is working.

↑ Throughput: Numbero of processes completed per unit of time.

↓ Turnaround time: Total time [waiting time + CPU exe. time

↓ Waiting time: Total waiting time in ready queue. [except I/O]

↓ Response time: Time for first respond.

TOPIC NAME : _____ DAY : _____

TIME : DATE : / /

FCFS → First come first serve

SJF → Shortest Job First

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

SRTF → Shortest remaining time first

Round Robin

Each p. gets a small unit of CPU time (time quantum)

No process waits more than $(n-1)q$ unit time.

q Large → FIFO.

q small → q must be large with respect to context switch, otherwise overhead is too high.

Priority Scheduling

Priorities are generally indicated by some fixed range of numbers. The process with the highest priority (not number) gets the cpu.

Number (small or big) that defines priority differs. Some consider the smallest = most priority.

Indefinit blocking/ starvation → A priority scheduling algorithm can leave some low priority processes waiting indefinitely.

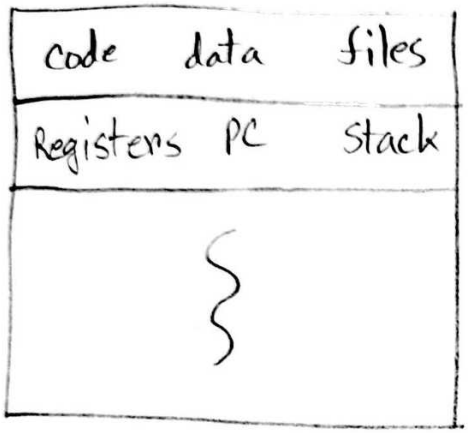
Aging → Gradually increasing the priority of a process that is waiting for a long time.

TOPIC NAME: _____ DAY: _____

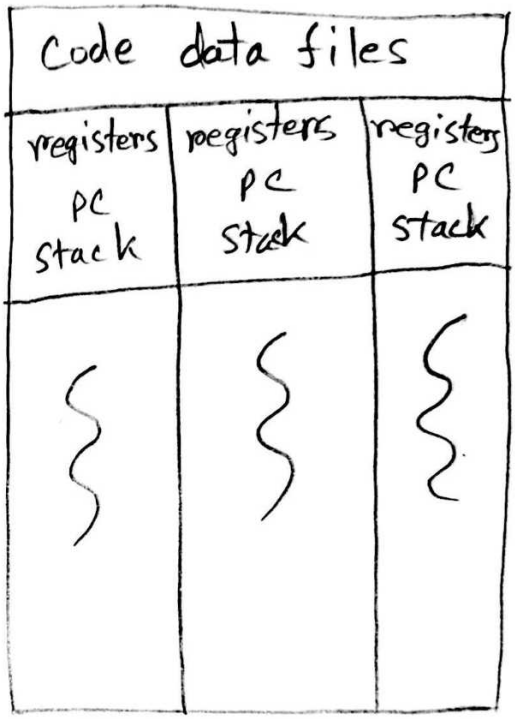
TIME: DATE: / /

Threads

A thread is a basic unit of CPU utilization.



single-threaded process



multi-threaded process

A process could divide into several threads and run in several core parallelly.

Threads share the same data and code but different stack and stuff, which results in less context switch.

k thread [PID=2] serves as the parent of all other kernel threads.

Benefits of threads

- Responsiveness
  One thread may provide rapid response while other threads are busy.
- Resource Sharing
  Resource sharing allows multiple task to be performed simultaneously in a single address space.
- Economy
  Creating and managing threads is much faster than doing the same for processes.
- Scalability
  single-threaded process only runs on one core but multi-threaded process runs on multi core.

Data parallelism

Distributes subset of the same data across multiple computing course and performing same operation on each core.

Task parallelism

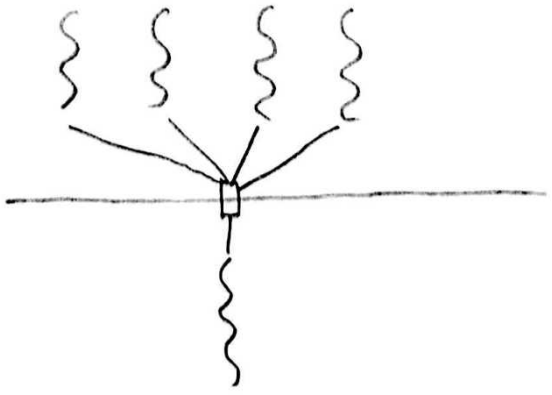
Distributes tasks (threads) across multiple cores, each thread perform a unique operation.

Multithreading Models

User threads are supported above the kernel and managed without kernel support.
kernel threads are managed and supported directly by the OS.

Many to one

Maps many user level threads to one kernel level threads. Thread management is done by thread library in user space.
The entire process will block if a thread makes a blocking system call. Because only on thread can access the kernel at a time.*) Multiple threads are unable to run on multicore system.



One to One

Each thread is mapped to a kernel thread.

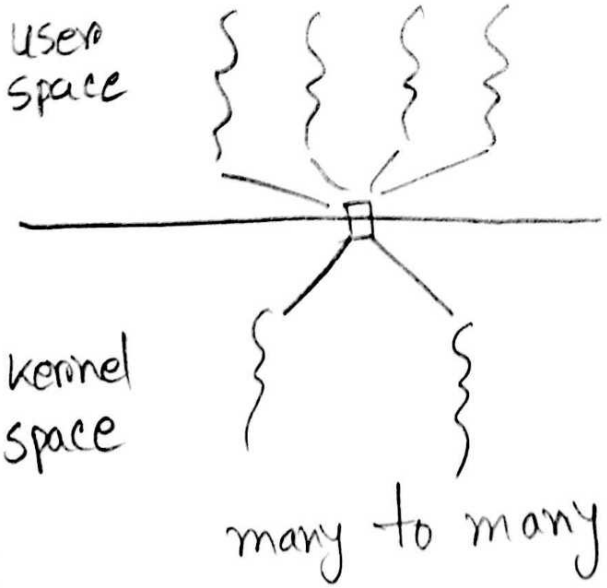
Another thread may run when a thread makes a blocking sys call. Allows multiple thread to run parallelly.

*) A large number of kernel threads may burden the performance.

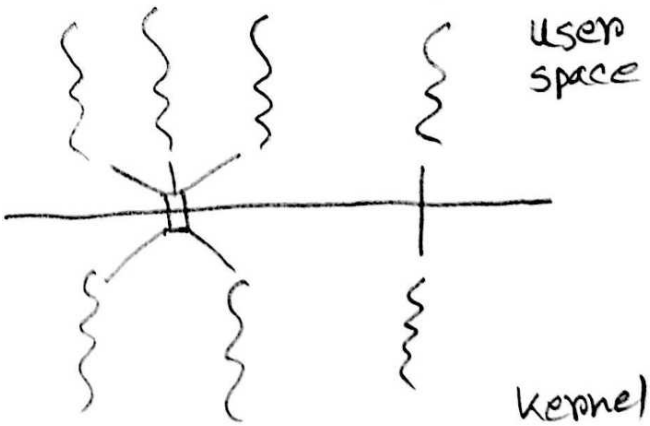
Many to many

Multiplexes many user level threads to a smaller or equal number of kernel threads.

*) Difficult to implement



many to many



Two level model

Asynchronous threading → Once parent creates a child, the parent resumes its execution. The threads are independent here. Very little data sharing.

Synchronous threading → Once parent creates one or more child, it must wait for all of its child to terminate before it resumes.

Implicit threading → The programmers does not create or manage threads directly. Instead, the compiler, runtime or OS creates and manages the threads.

i. Thread creation and destruction are time consuming.

ii. Creating unlimited threads can exhaust system resources.

Sln → Thread pool

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Thread pool

A group of threads created before they are needed.

They stay idle, waiting for work.

i. Faster execution.

ii. Limits the number of threads.

iii. Flexible task scheduling. The task could be execute after a time delay or to execute periodically.

4.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Threading Issues

fork() → creates a duplicate child process.

exec() → replaces the child process with given process.

fork() is of two types

- → duplicates all thread (T_1)
- → child is single threaded (T_2)

exec() is called immediately after fork() → T_2

exec() is not called → T_1

5.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Signal Handling

Signals are used in UNIX system to notify a process that a particular event has occured.

A signal handler is used to process signals. All signals follow the same pattern,

i. A signal is generated by occurence of a particular event.
ii. The signal is delivered to a process.
iii. Once delivered, the signal must be handled.

Synchronous signal → A running process performs illegal memory access or division by 0. These signals are delivered to the same process that performed these.

Asynchronous signal → Signal is generated by an event external to a running process. Typically, this signal is sent to another process.

Two possible handlers,

i. Default signal handler

ii. Users defined signal handler (Default one gets replaced)

Delivering Signals

- → Deliver to the thread to which the signal applies.
- → Deliver to every thread in the process.
- → Deliver to certain threads in the process.
- → Assign a specific thread to receive all signals for the process.

Thread Cancellation

Terminating a thread before it has completed.

Thread to be canceled is target thread.

Two scenarios:

i. Asynchronous cancellation → One thread immediately terminates the target thread.

A thread may be stopped before it finishes its work. May be cancelled while updating a data shared with other threads. Often, OS reclaims system resources from that thread but will not reclaim all resources.

ii. Deferred cancellation → Allows the target thread to periodically check if it should be cancelled.

Cancellation occurs only after the target thread has checked a flag to determine if it can be safely cancelled or not.

Process Synchronization

It involves using tools that control access to shared data to avoid race condition.

↓

situation where several processes manipulate data at the same time and the outcome depends on order in which the access take place.

Critical Section → Segment of code of each process which may change or access shared data.

```
while (True){                    ← buffersize
while (c == B_size);
buffer[in] = next_produced;
in = (in+1) % B_size;
c++;}
```

```
while (True){
while (c==0);
next_consumed = buffer[out]
out = (out+1) % B_size
c--;}
```

register

$r_1 = c$
$r_1 = r_1 + 1$
$c = r_1$

$r_2 = c$
$r_2 = r_2 - 1$
$c = r_2$

Data integrity problem

Requirements of solution to critical section problem:

i. Mutual exclusion: Only one process can be executing in its critical section.

ii. Progress: If nobody in critical section and some process want to enter it, one of them must be allowed to enter. The selection can not be postponed indefinitely.

iii. Bounded waiting: There must be a limit on how many times other processes can enter the critical section before a waiting (requested) process gets its turn.

Preemptive kernel → Allows process to be preempted while it is running in kernel mode.

Nonpreemptive kernel → Doesn't allow processes running in kernel mode to be preempted.

Hardware Based Solution

Many computer provide special hardware 'instructions' that allows to test and modify the content of a word or to swap the contents of two words atomically.

Test and set lock

shared var Lock { 0 unlocked, 1 locked }

Before entering a cs a p. inquires about Lock.

```
boolean test_and_set(boolean *target){      // makes a pointer
    boolean rv = *target;                   // value of add.
    *target = TRUE;                         // gets the value of address target
    return rv; }
```

loops & does nothing

```
do{ while test_and_set(&lock));             // gets the add. of lock
    /* cs */
    lock = false;
    /* remainder */ } while (true);
```

compare and swap

```
int com_and_swa(int *value, int expected, int new){
int temp= *value;
if (*value == expected){
    *value = new;}
return temp;}
```

```
do{ while com_and_swa(&lock, 0, 1) != 0);
    /* cs */
    lock = 0;
    /* remainder */ } while (true);
```

TIME: DATE: / .

Mutex lock

↳ Mutual exclusion

```
while (true){
    acquire lock
      /* cs */
    release lock
      /**/}
```

*) has a var available

Disadvantage

Busy waiting: If a p. is in cs, any other p. trying to enter cs will loop continuously.

```
acquire(){
  while (!avaiable);
  available =false;}
```

```
release(){
  available = true;}
```

TOPIC NAME : _____ DAY : _____

TIME : DATE : / /

contended lock → Locked situation. A thread gets blocked while trying to acquire the lock.

uncontended lock → unlocked situation.

*Spinlocks are used when the lock is to be held for a short duration. Use spinlock if the lock will be held for less than two context switches.

spinlock → A waiting thread keeps checking (looping) the lock repeatedly instead of going to sleep.

Semaphore

→ an integer S.

↓

accessed only through two standard atomic operations: wait(), signal()

P V

```
wait(S){
    while (S<=0);
    S--;}
```

```
signal(S){
    S++;}
```

→ busy waiting

*) atomic operations are executed without interruption.

Counting semaphore → 'S' can range over an unrestricted domain.

binary semaphore → 'S' can only be 0 or 1.
Behave similarly to mutex lock.
Solve sync. problems.

*) C. Semaphore can be used to control access to a given resource consisting of finite number of instances.

solving sync. problem with binary semaphore

Let, $S_1 \in P_1$,
$S_2 \in P_2$.

S_1 needs to run before S_2

```
S=0

P1:                    P2:
S1;                    wait(S);
signal(S);             S2;
```

Semaphore implementation

We are trying to stop busy waiting. In this way semaphore value may be negative.

```
typedef struct{
        int value;
        struct process *list;
} semaphore;
```

struct creates a structure with some data.
Here it is named semaphore, keeps an int and a list of process structure.

```
wait(semaphore *s){
        s->value--;
        if(s->value<0){
            add this process to s->list;
            sleep();
        }
}
```

think it like → s.value--

sleep(); → suspend the P. that invokes it
is a sys. call

```
signal(semaphore *S){
    S->value++;
    if (S->value <=0){
        remove a process P from S->list;
        wakeup(P);
    }
}
```

wakeup(P); → resumes the suspended process "P"
is a sys.call

P gets the resources freed by the process that called signal().

It must be guaranteed that no two p. can execute wait() or signal() on the same semaphore at the same time.

In single core it can be done by disabling interrupt.

In multicore interrupts must be disabled on every core. This can seriously diminish performance.

*) Busy waiting have not been completely eliminated. Rather it is moved to the critical section.

Before [while (s<=0);] created busy waiting now when s<0 the p. is put to sleep.

But we have to protect the 's' value inside wait() and signal(). We have limited 'Busy waiting' to the 'cs' of this two programs.

Deadlock

A situation where two or more processes are waiting in the queue for an event that can be caused only by one of the waiting processes. Here these two processes depends on each other.

```
P0                                  P1
wait(S); -> P0 gets S               wait(Q); -> P1 gets Q
wait(Q); -> P0 wants Q              wait(S); -> P1 wants S
   .                                   .
   .                                   .
   .                                   .
signal(S); -> P0 releases S         signal(Q); -> P1 releases Q
signal(Q);                          signal(S);
```

Starvation -> A p. keeps waiting for a resource for a very long time because other processes keep getting the resource before it.

Priority Inversion

Assume three P. with priority L < M < H.

H requires semaphore 'S' used by L. Ordinally H would wait for L to finish using resource S. Now, M becomes runable, thereby preempting L.

Indirectly, a process with lower priority, process M, has affected how long H must wait for L to relinquish resource S.

It can occur only in systems with more than two priorities.

Priority inheritance protocol

P. that are using resources needed by a higher priority P. inherit the higher priority. After they are finished with the resources, the priority revert to the original value.

Here, L will temporarily inherit priority of H and prevent M from preempting it. After using S priority will go back to normal and H would run next.

Bounded Buffer Problem

concern 1 → Producer won't produce data if buffer is full

concern 2 → Consumers " remove " if buffer is empty.

Semaphore

mutex = 1 → binary semaphore

c_full = 0

c_empty = Buffer size } counting semaphore

```
producer(){
    while (True){
        item = producedItem();
        wait(c_empty);
        wait (mutex);
        put_in_buffer(item);
        signal (mutex);
        signal (c_full);
    }
}
```

```
consumer() {
    while(True){
        wait (c_full);
        wait (mutex);
        item = rm_from_buffer();
        signal (mutex);
        signal(c_empty);
        consumeItem(item);
    }
}
```

Reader Writer Problem

Two readers can access a data without any issue.

If a writer and some other process access the data at the same time, problem may occur.

First variation → Readers won't wait for other readers.

Second variation → If a writer is ready, it will perform as soon as possible. No new readers can start reading.

for First suppose A, B is reading and writer C is waiting. After some times D comes and start reading forcing writer to starve.

for second reader may starve.

rwLock is most useful

i. when it is easy to identify readers and writers
ii. more readers than writers.

First soln (reader writer lock)

rw_mutex = 1 → shared by both (lock for write check)

mutex = 1 → lock for reader count

} semaphore

int read_count = 0

Writer

```
while (True){
    wait(rw_mutex);
    /* writes*/
    signal (rw_mutex);
}
```

takes/locks write access for only one writer

i. Only one writer

ii. Many readers

iii. Many readers can aquire rw_mutex

Reader

```
While (True){
    wait (mutex);
    read_count ++;
    if (r_c == 1){
        wait (rw_mutex);}
    signal (mutex);
    /* read */
    wait (mutex);
    r_c --;
    if (r_c == 0){
        signal(rw_mutex);}
    signal (mutex);
}
```

Dining philosophers Soln

One solution is 5 semaphore for 5 chopsticks

```
semaphore chop[5];

while (True){
    wait(chop[i]);
    wait (chop[(i+1)%5]);

    /* eat */
    signal (chop[i]);
    signal ( chop[ (i+1)%5]);

}
```

If all philosopher takes the left chopstick dead lock will arrive.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Possible Soln

i. Allow 4 philosopher instead of 5 at a table.

ii. Allow philosophers to pick chopstick if both chopsticks are available.

iii. Use asymetric solution- odd ph. picks left chop first, even ph. picks right chop first.

File System Concept

Determines how user data or files gets stored and fetched in the secondary memory.

In every OS data from user end is managed in form of files.

File system divides each file into multiple equal sized blocks named logical blocks.

$2\ (2^n)$

logical block size = physical block/sector size

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Allocation Methods

Contiguous

Linear allocation, after b block, next file would be stored in b+1 block

Non Contiguous

Linked

Something like linked list.

Solves external Fragmentation.

Can't directly access data.

Indexed

Unix inode

Indexed Allocation

Uses one block to store all the pointers of the linked blocks, called index block. No external fragmentation

problem: max size is an issue.

Soln

- linked scheme
  To allow large files, several index blocks can be linked together.

- Multilevel Index
  First level index block point to second level index blocks.

- combined
  some pointers in index block point to direct data some to index blocks.

Inode
- 12 pointers → direct data
- 3rd Last → single indirect block
- 2nd " → double " "
- Last → tripple " "

Memory Management

For proper system operation we must protect the OS from access by user processes and also protect user processes from each others.
The OS doesn't usually intervene between CPU and its memory access. So this protection must be provided by the hardware.

First we need to make sure each process has a separate memory space. This is done by using two registers. Base and limit.
controlled by OS
Any attempt by a program executing in user mode to access OS memory or other users' memory results in a trap to the OS.

Logical vs Physical Address Space

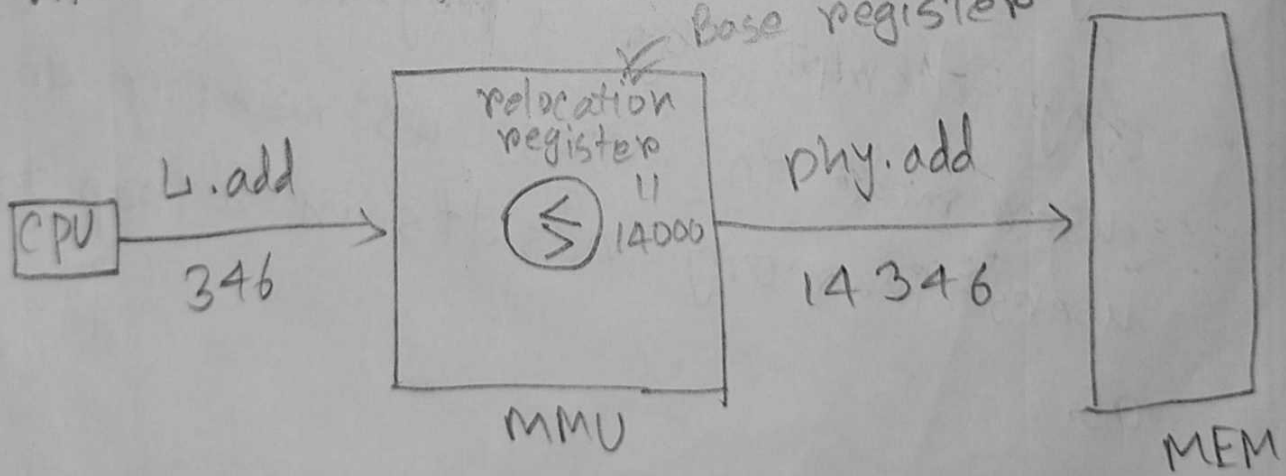
↓ generated by CPU

Logical Add. Space
set of all logical addresses generated by a program.

mem. add. register has physical memory

Physical Add. Space
set of all physical add. corresponding to these logical addresses.

Mem. Management unit does mapping of virtual to physical mem. add.



Paging

Makes physical add. noncontiguous.
Avoids external fragmentation.
Still have internal fragmentation.

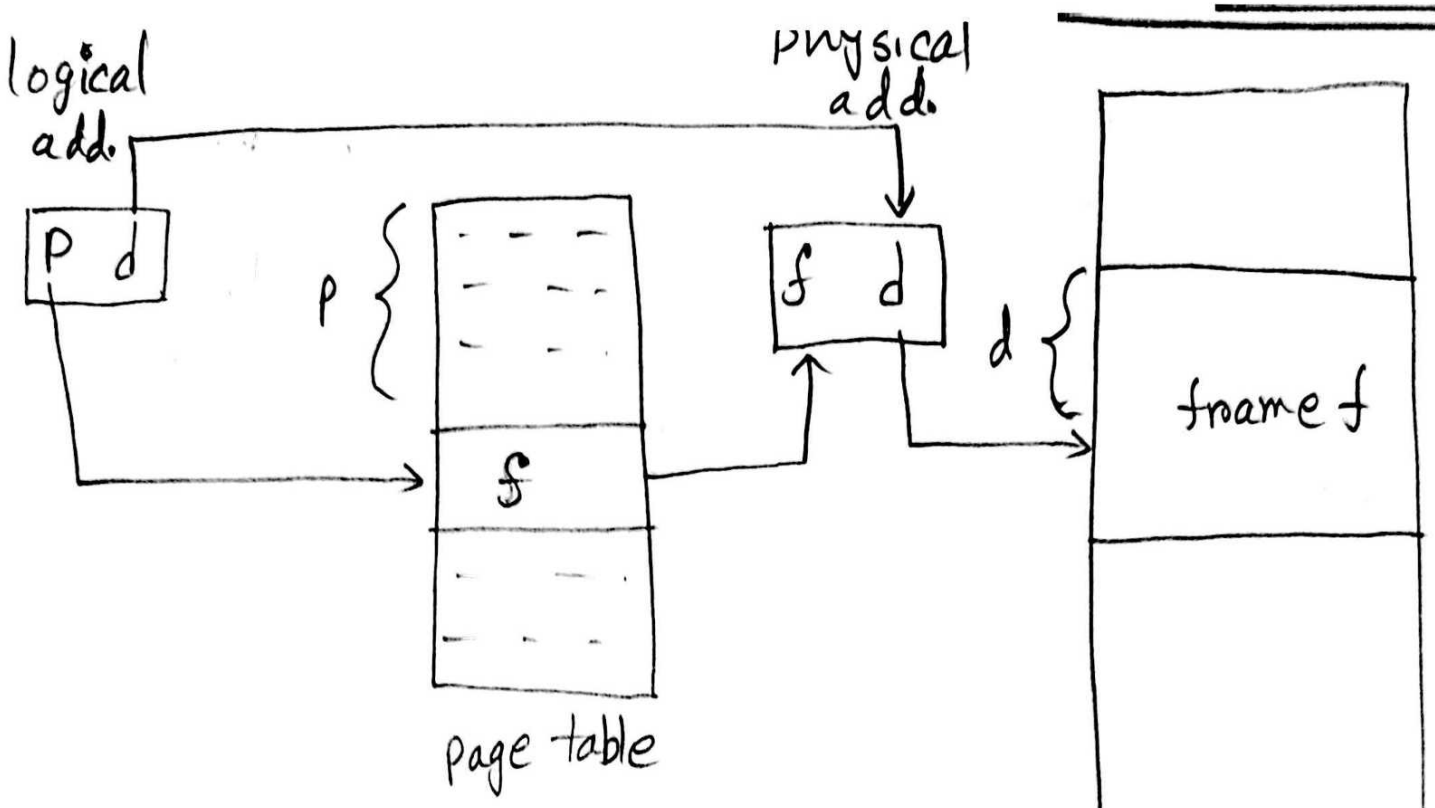
Basic Method

Breaking physical mem. into fixed size called frames
" Logical " " " " " " pages

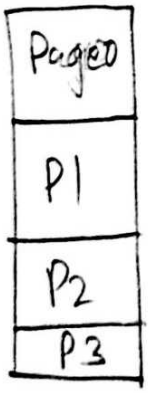
Frame size = Page size

When a p. is to be executed its pages are loaded into available mem. frames.

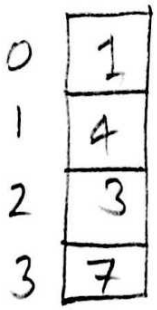
Add. generated by the CPU has two parts.
page number P
page offset d



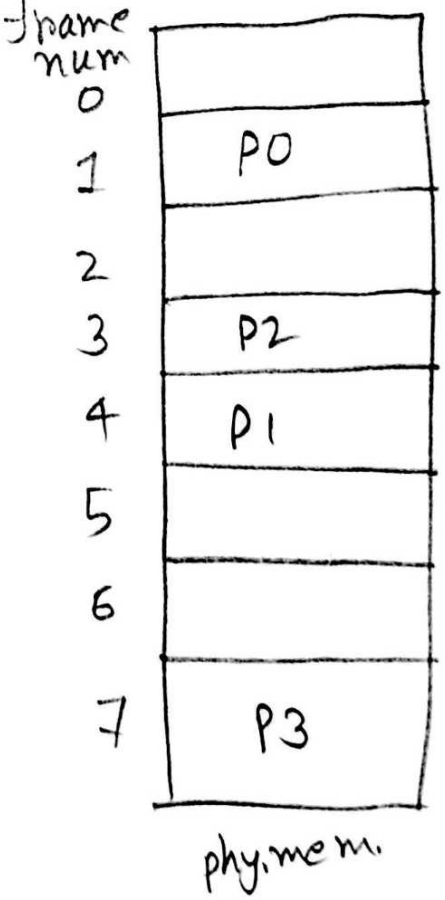
page table contains the base add of each frame in phy. mem.



logical mem.



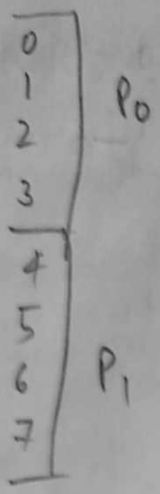
page table

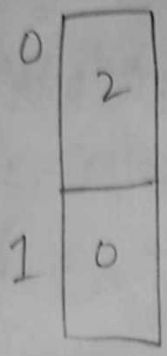


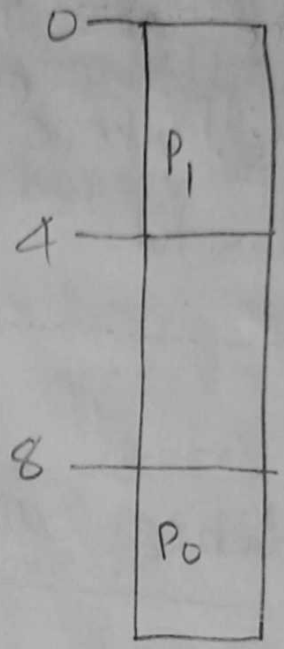
phy. mem.

TOPIC NAME : _____ DAY : _____

TIME : DATE : / /







If process size is independent of page size then average internal fragmentation will be on average one half page (last page).

So making page size smaller will be desirable.

But there is overhead in each page table entry.

User p doesn't address memory outside of its page table.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

. Frame table: Has one entry for each physical page frame. Where to which process that frame belongs to.

Hardware Support

Each process has its own page table, a pointer to that page table is stored in the process's PCB. During context switch CPU must load the new process table.

First method could be keeping the entire page table in hardware registers. Which makes the translation very efficient. But increases context switch time. As each of thes registers must be changed. If the table size is huge this method wont be useful.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

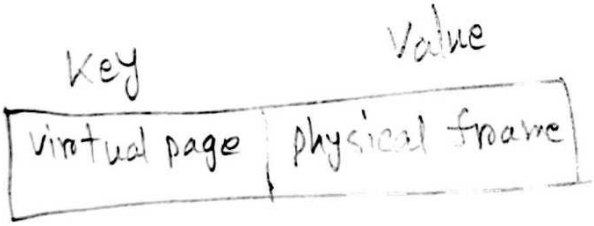
Rather, the page table is kept in main mem and a page table base register (PTBR) points to the page table. Context switch requires only changing one register.

But it may result in slower access time.

This method requires two mem. access.

One for the page table and one for the data.

This problem can be solved by the use of a special fast look up hardware cache called "associative memory" or "translation lookaside buffer" (TLB)



Associative memory compares the requested key with all stored keys at the same time.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

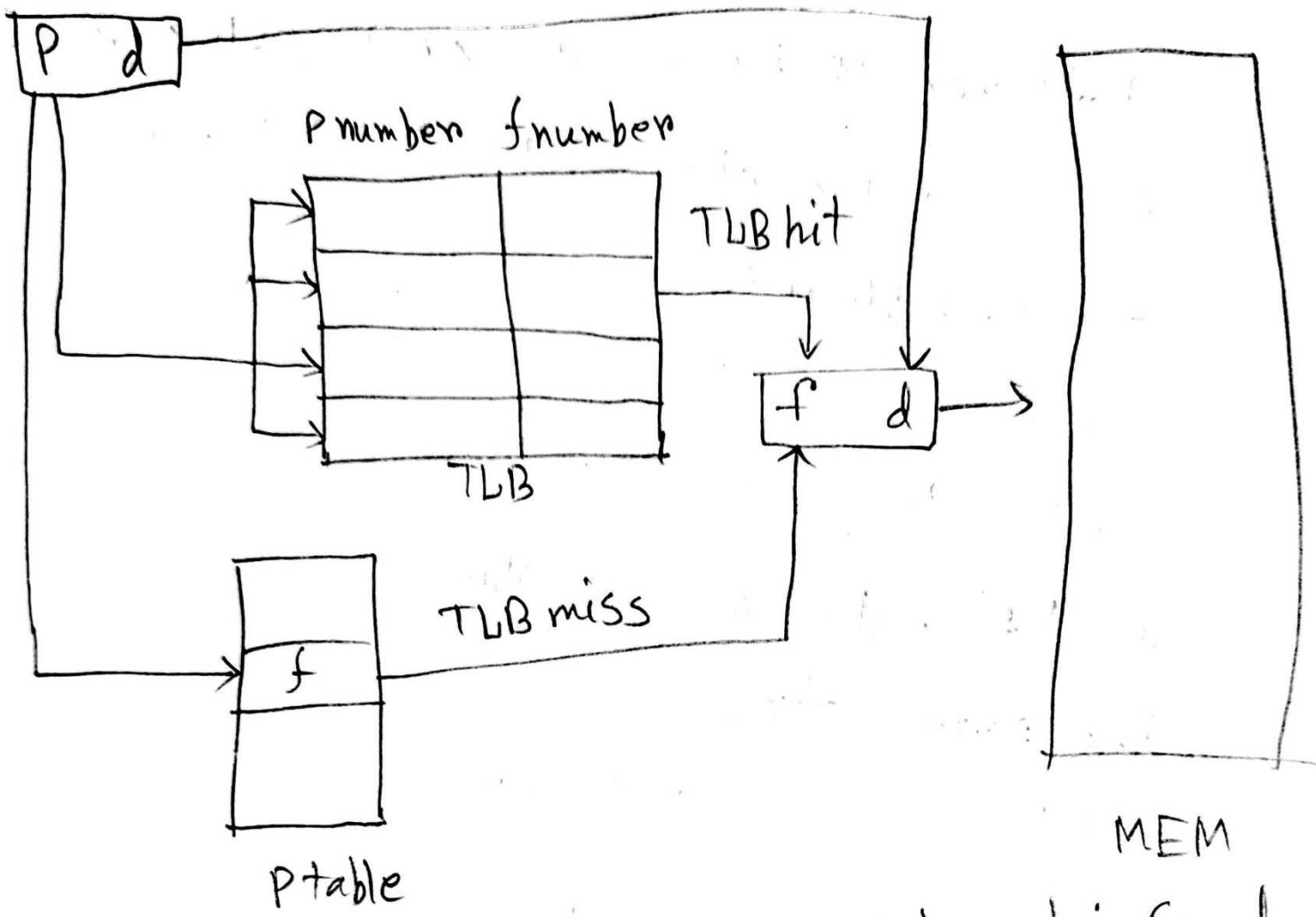
TLB contains only a few page of the page table, when CPU generates a logical add MMU first checks if its page number is present in TLB. If the page number is found then the frame number is immediately available. If the page number is not in TLB (TLB miss) MMU looks up the page table in RAM.

TLB entries for key kernel code are "wired down" meaning they can not be removed.

ASID (Address Space Identifier) -> unique ID for each P.

Some TLB stores ASID and used for address space protection for that P.

If TLB doesn't support separate ASID then in every context switch TLB must be cleaned.



hit ratio → % of page number of interest is found in TLB.
α

Memory Protection

Implemented by associating protection bit with each frame to indicate read-only and read-write access is allowed.

Any violations result in a trap to the kernel.

A valid-invalid bit is attached to each entry in the page table.

valid → the associated page is in the process's logical address space.

Shared Pages

Reentrant → read only code

↳ shared among processes

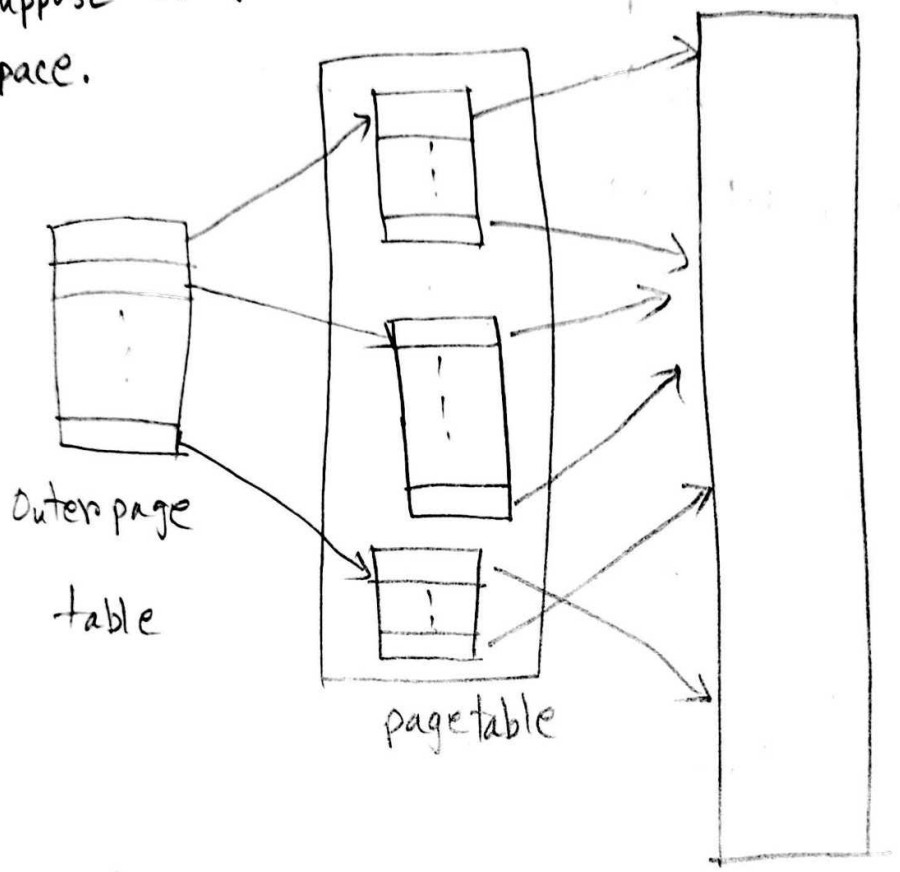
every P. has page table with reentrant code page but it is loaded as frames in physical memory only once.

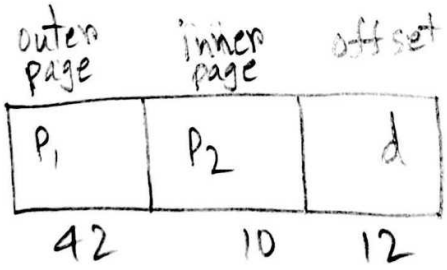
TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Hierarchical

Suppose a system with 64 bit logical address space.



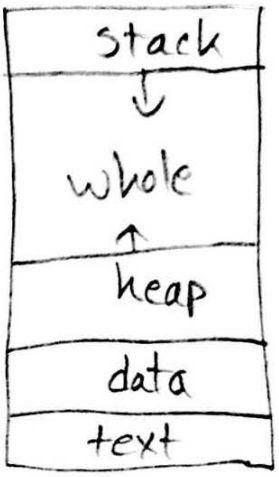


known as "forward mapped page table".

Virtual Memory

A technique that allows the execution of processes that are not completely in memory. Each p. thinks it has a large, private memory.

Virtual address spaces with wholes are known as "sparse" address spaces.



Virtual add. space of a p. in ram/rom

Demand Paging

↳ load pages only when they are needed.

valid-invalid bit is used here,

valid ⟶ page is legal and in memory

invalid ⟶ page is not legal or not in main memory.

Marking a page invalid will have no effect if the p. never tries to access that page.

But if a p. tries to access page that was not brought into memory, it causes a "page fault". This will cause a trap to the OS.

pros.

Less I/O needed

" mem. "

Faster response

More users

cons

page fault interrupt

Page fault handling

→ check internal table and determine valid/invalid mem. access.

→ If invalid terminate the P. else now we page it in.

[If no free frame available then replace a page]

→ We find a free frame.

→ schedule a secondary storage operation to read the desired page into the frame.

→ modify internal table and page table to indicate that the page is now in memory.

→ restart the instruction.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

pure demand paging: never bring a page into mem. until it is required. No pages are loaded in memory when a p. starts. First a page faults happen and then the first page comes.

some programs could cause multiple page fault per instruction. Programs tend to have "locality of reference" which results in reasonable performance from demand paging.

→ a program tends to access the same or nearby mem. locations repeatedly over a short period of time.

FiFo

ref → 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

f_3				1	1	1	4	4	4	4	4	2
f_2		3	3	3	3	2	2	2	2	2	5	5
f_1	2	2	2	2	5	5	5	5	3	3	3	3
	*	*	hit	*	*	*	*	hit	*	hit	*	*

hit = 3
pf = 9

f_3			3	3	3	2	2	2	2	2	4	4			
f_2		2	2	2	1	1	1	1	1	3	3	3			
f_1	1	1	1	4	4	4	5	5	5	5	5	5			
	*	*	*	*	*	*	*	hit	hit	*	*	hit			

hit = 3
page-fault = 9

ref → 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

f_4				4	4	4	4	4	4	3	3	3
f_3			3	3	3	3	3	3	2	2	2	2
f_2		2	2	2	2	2	2	1	1	1	1	5
f_1	1	1	1	1	1	1	5	5	5	5	4	4
	*	*	*	*	hit	hit	*	*	*	*	*	*

hit = 2
pagefault = 10

Belady's anomaly: for some page replacement algorithms the page fault rate may increase with the number of allocated frames.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Optimal [Stack Algorithms]

- lowest page fault rate.
- will never suffer from Belady's anomaly.

"Replace the page that will not be used for the longest period of time in future."

ref → 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

f_3				1	5	5	5	5	5	5	5	5
f_2		3	3	3	3	3	3	3	3	2	2	2
f_1	2	2	2	2	2	2	4	4	4	4	4	4
	*	*	hit	*	*	hit	*	hit	hit	*	hit	hit

hit = 6

pfault = 6

LRU (Least Recently Used) [Stack Algorithms]

ref → 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

f_3				1	1	1	4	4	4	2	2	2	hit=5
f_2		3	3	3	5	5	5	5	5	5	5	5	pf=7
f_1	2	2	2	2	2	2	2	2	3	3	3	3	
	*	*	hit	*	*	hit	*	hit	*	*	hit	hit	

*) Doesnot suffer from Belady's anomaly.

Implementation

→ Counter
page tables must have a time field, cpu has a counter. when cpu access a page that counter increases and current value is saved on time field of that page.
problem: system must update time for each mem. access. Also need to search the PT for the smallest time.

→ Stack: keep the pages in stack and remove the bottom page.

Protection

Involves controlling the access of p. and users to the resources defined by a computer system.

Principle of Protection

i. Principle of least Privilege ✱
Give a user or program only the permission it needs to do its job and nothing more.

ii. Not using root all the time
In UNIX, root has almost unlimited power.
So, normally users should not run programs as root.

iii. Compartmentalization
Divide the system into separate protected parts, each with its own permissions. Ex: Separating public web server from the internal database.

iv. Audit Trail

It is a hard record in the system logs.

v. Defense in Depth

Using multiple layers of protection.

Protection rings

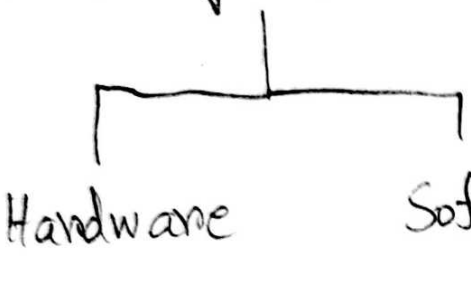
- A popular model of privilege separation of processes.
- higher privilege level processes get permission to perform more powerful operations that lower privilege processes cannot.

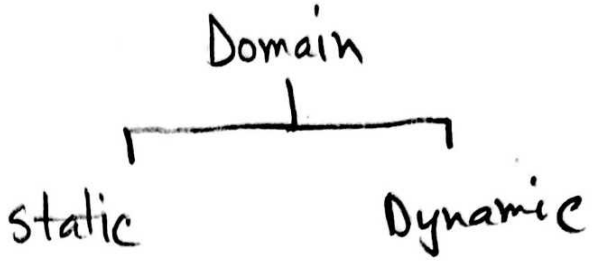
Policy → Defines what states are allowed.

Mechanism → What is built into the OS for its protection.

Domain of Protection

A computer system is a collection of processes and objects.





Objects have unique names and defined operations.

Domain is a set of permissions that tells a p. what it can access and what operations it can perform.

Domain is a collection of access rights, each of which is an ordered pair < object-name, rights-set>

Each user, process or procedure can be domain.

Domain Implementation in UNIX

Domain = User ID

Each file has a domain bit "setuid bit"

When file is executed and setuid = on then user id is set to owner of the file being executed.

When execution is complete userid is resetted.

command

su ⟶ switch to another users domain. (Pass. needed)

sudo ⟶ Run a specific comand in another domain.

Domain Implementation in Android App IDs

In Android, distinct user IDs are provided on a per-application basis.

After installing an app the installd daemon gives it an unique userID and groupID, along with a private data directory whose ownership is granted to this UID/GID combination alone.

Certain operations could be allowed ony to members of a particular GID.

Android can also define some UIDs as isolated. An isolated process has very limited access to system services.

Access Matrix

access control list for an object

object / domain	F_1	F_2	F_3	Printer
D_1	read		read	
D_2				print
D_3		read	exe	
D_4	r&w		r&w	

Access(i,j) is the set of operations a p. executing in domain(i) can invoke on object(j).

user who creates object can define access column for that object.

The access Matrix can also allow permissions to change.

copy -> can copy an access write to another object.

control -> A domain can modify another domains access right.

transfer -> A p. can switch from one domain to another.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

policy → user decides who can access what object in what mode.

mechanism → OS checks access matrix before a p. does anything.

Access matrix can't solve the general confinement problem.

↳ when a program leaks information.

suppose a p. can read a secret file, after reading it could send the info elsewhere.

object / domain	D_1	D_2	D_3	D_4
D_1		switch		
D_2			switch	switch control
D_3				
D_4	switch			

here, domains are in object form too.

*) A p. in D_2 could modify access rights of D_4.

A p. in D_1 can switch its domain to D_2
But a p. in D_2 can't switch its domain to D_1

object / domain	F_1	F_2	F_3
D_1	exe		write*
D_2	exe	read*	exe
D_3	exe		

D_1 can copy the write operation to other domains

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Revocation of Access Rights

The access list is searched for any access rights to be revoked and they are deleted from the list. It could be immediate or delayed, selective or general, partial or total, temporary or permanent.

Identify capabilities before revoking them.

Reacquisition → permission ticket

Try to reacquire after deletion.

Back pointers

Objects has pointers pointing to all capabilities associated with that object. During revocation following these pointers capabilities could be changed. Used in MULTICS. Costly.

Indirection

capability points to a global table, which in turn points to the object. Used in CAL system. Doesn't allow selective revocation.

↳ revoking for selective users.

key

Each object has a master key.
when capabilities are created the current value of master key is given.
when capability is used both key needs to match.
Revocation replaces the master key.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Mandatory Access Control

Directionary Access Control

- Access is based on users/groups
- owner can decid who can access
- If hacker gets root access, they may access protected resources.

MAC

- Access is controlled by a system wide security policy.
- even root normally can't bypass that policy.
- system decides who can access what.

MAC uses labels for users/processes and objects.
(users/processes = subject)

When a subject req. access to an object, policy checked to determine if given label holding subject is allowed or not.

Linux's capabilities slices up root powers into distinct areas, each represented by a bit in bitmap. Control over privileged operations can be achieved by toggling bits in the bitmap.

Three sets of bitmaps - permitted, effective and inheritable

- → can apply per process or per thread.
- → once revoked, can not be reacquired.
- → a p. or t. starts with full set of permitted capabilities. It may have all the capabilities it is allowed to have at first. During execution it can give up capabilities it doesn't need.

capabilities are direct implementation of the principle of least privilege.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

System integrity protection

Even root can not modify protected OS components. Only authorized/ code-signed programs can perform certain privileged operations.

System call Filtering

Restrict what a p. can ask the kernel to do. Each process can be allowed only a safe subset of sys. call. Arguments passed to sys. call are also checked.

Sandboxing

Restrict a p. to a limited environment. A p. only gets resources it needs.

Code signing

Varify that a program is authentic.
Uses cryptographic signature.
OS refuse to run p. with invalid signatures.

Language Based Protection

Allows high level description of policies for the allocation and use of resources.

If hardware doesn't directly suppor the required protection the language can help enforce it.

Converts high level protection rules into sys. call.

TOPIC NAME: _____ DAY: _____

TIME: DATE: / /

Security

Security is a measure of confidence that the integrity of a system and its data will be preserved.

Violations

Breach of confidentiality: Unauthorized reading of data

Breach of Integrity: Unauthorized modification of data

Breach of availability: Unauthorized destruction of data

Theft of service: Unauthorized use of resources.

Denial of service: Prevention of legitimate use.

Security Violation Methods

Masquerading: Pretending to be an authorized user.

Replay attack: Fraudulent repeat of valid data transmission.

Man in the middle: Intruder sits in data flow.

Session hijacking: Intercept an already established session.

Privilege escalation: common attack type with access beyond what a user or resource is supposed to have.